

Algorithms & Java Benchmarking — Analytical Report

Setup

- $N = 1,000,000$ 32-bit integers (full scale, except where noted for bubble sort).
- JMH 1.37, AverageTime mode, results in ms/op, 1 fork.
- Each `@Benchmark` method sorts a fresh in-memory copy of the dataset (`@Setup(Level.Invocation)`), so only the sort call itself is timed, not the copy.
- Distributions: uniform random, sorted ascending, reverse sorted descending, nearly sorted (~1% random swaps, i.e. ~10,000 swapped pairs at $N = 1,000,000$).
- Machine: single run on the development machine (not an isolated benchmarking server); absolute numbers will vary by hardware, but the relative ordering and asymptotic behavior are the point of this report.
- Iteration counts used to produce the numbers below were reduced from the assignment's suggested 3–5 warmup / 5–10 measurement to 2 warmup / 3 measurement (2s each) to keep the full run inside a few minutes. The benchmark class itself is still configured with the assignment's suggested defaults (`@Warmup(iterations = 3)`, `@Measurement(iterations = 5)`); a full run with `java -jar target/benchmarks.jar` uses those.

Measured Results (QuickSort, RadixSort, Arrays.sort, $N = 1,000,000$)

Algorithm	Random (ms)	Sorted (ms)	Reverse (ms)	Nearly Sorted (ms)
QuickSort	75.7 ± 15.2	5.6 ± 0.1	26.8 ± 7.1	7.8 ± 0.04
RadixSort	4.3 ± 0.1	13.9 ± 0.4	13.9 ± 0.3	13.7 ± 0.1
Arrays.sort	53.0 ± 11.6	0.19 ± 0.01	0.64 ± 0.01	14.2 ± 3.1

Bubble Sort: Measured + Extrapolated

Bubble sort's $O(n^2)$ worst case makes a full JMH run (warmup + measurement iterations) impractical at $N = 1,000,000$ for any distribution other than already-sorted (where early-exit terminates after a single $O(n)$ pass).

Measured directly (sorted distribution, same JMH settings as above):

Algorithm	Sorted (ms)
BubbleSort	0.19 ± 0.01

Extrapolated for the other three distributions: a standalone single-call timing probe (not JMH, no warmup) measured bubble sort at $N = 2,000 / 5,000 / 10,000 / 20,000$:

N	Random (ms)	Reverse (ms)	Nearly Sorted (ms)
2,000	7	5	4
5,000	32	34	26
10,000	60	87	38
20,000	361	351	152

Scaling the N = 20,000 figures quadratically (bubble sort is $O(n^2)$, so cost scales by $(1,000,000 / 20,000)^2 = 2500\times$) gives **single-call** estimates at N = 1,000,000:

Distribution	Estimated single-call time
Random	≈ 15 minutes
Reverse	≈ 14.6 minutes
Nearly Sorted	≈ 6.3 minutes

These are order-of-magnitude estimates from an un-warmed JVM, not JMH-quality measurements — but they explain why a full automated JMH run (3+ warmup, 5+ measurement iterations, each needing its own sort call at this scale) was not executed for these three distributions: it would take on the order of 1.5–2 hours just for bubble sort.

Discussion

- **RadixSort wins decisively on random data** (4.3ms vs. 53–76ms) because it is $O(n \times k)$ for a fixed key width k (4 byte-passes here), with no data-dependent branching — its cost is essentially distribution-independent (4.3–13.9ms across all four distributions, the tightest spread of any algorithm tested).
- **QuickSort and Arrays.sort (dual-pivot quicksort) are both extremely fast on sorted/reverse-sorted input** (0.19–26.8ms) because median-of-three pivot selection (QuickSort) and the JDK's own pivot strategy avoid the classic worst case that a naive first/last-element pivot would hit on these exact inputs. Reverse-sorted is still QuickSort's worst distribution here (26.8ms) — consistent with median-of-three reducing, but not eliminating, the partition asymmetry on monotonic input.
- **RadixSort is, perversely, slowest on sorted/reverse/nearly-sorted data relative to its own random-case time** (13.7–13.9ms vs. 4.3ms) because it always performs the same fixed amount of work (4 full passes over all 1,000,000 elements) regardless of existing order — it cannot exploit partial sortedness the way comparison sorts can.
- **Bubble sort is the only algorithm in this set that is asymptotically worse than the others**, and the difference is not subtle: where every other algorithm finishes in milliseconds at N = 1,000,000, bubble sort needs minutes-to-hours on anything but already-sorted input. This is the central lesson of the assignment — algorithmic complexity, not micro-optimization, dominates at this scale.
- **Arrays.sort is fastest on sorted/reverse-sorted input** (0.19ms / 0.64ms) — faster than the hand-written QuickSort on the same inputs (5.6ms / 26.8ms). The JDK's dual-pivot quicksort (with a fallback to a different algorithm for nearly-sorted runs) is more heavily tuned than this assignment's straightforward median-of-three implementation, which is the expected and correct outcome: the JDK sorter is the "reference" sorter for a reason.

Correctness Verification

All four algorithms are verified against `Arrays.sort()` using the assignment-provided `SortVerifier.assertCorrect()` (checks both `isSorted()` and element-for-element equality with the JDK-sorted reference), across all four distributions plus edge cases: empty array, single element, duplicate-heavy arrays, all-equal arrays, and integer boundary values (`Integer.MIN_VALUE`, `Integer.MAX_VALUE`). See `src/test/java/com/algorithms/sort/` — 17 tests, all passing (`mvn test`).